Rick & Morty Universe

Documentación del proyecto

Actividad de Aprendizaje 1 — 1ª Evaluación
2º DAM — Desarrollo de Interfaces
Curso 2025-2026

Marcos Martínez Vijuesca

1. Descripción de la aplicación

Nombre: Rick & Morty Universe

Rick & Morty Universe es una aplicación web interactiva desarrollada con React y TypeScript que permite explorar el universo de la serie de animación Rick & Morty. La aplicación consume la API pública de Rick & Morty (https://rickandmortyapi.com) y ofrece al usuario una experiencia de navegación fluida e intuitiva a través de sus personajes, ubicaciones y episodios.

2. Funcionalidades implementadas

2.1 Funcionalidades obligatorias

Consumo de 3 endpoints de la API (Punto 1)

La aplicación consume tres endpoints distintos de la API de Rick & Morty, cada uno con manejo de estados de carga y error:

- **/character** — Lista y detalle de personajes
- **/location** — Listado de ubicaciones
- **/episode** — Listado de episodios

Cada llamada a la API gestiona tres estados: carga (mostrando un spinner con mensaje), error (mostrando un mensaje descriptivo en rojo) y éxito (mostrando los datos). Esto se implementa mediante los hooks useState y useEffect de React.

Navegación y routing dinámico (Punto 2)

Se utiliza React Router DOM para la navegación entre vistas. Las rutas definidas son: / (Inicio), /characters (Personajes), /character/:id (Detalle de personaje con ruta dinámica), /locations (Ubicaciones) y /episodes (Episodios). La ruta /character/:id utiliza el parámetro id para hacer una llamada específica a la API de detalle. El componente Navigation aparece de forma consistente en todas las vistas al estar incluido en App.tsx fuera del sistema de rutas.

Búsqueda, filtrado y ordenación (Punto 3)

Las páginas de Personajes, Ubicaciones y Episodios incluyen el componente SearchBar que permite: búsqueda por texto en tiempo real sobre el nombre, filtrado por una propiedad (estado del personaje, tipo de ubicación o temporada del episodio) y ordenación ascendente (A→Z) o descendente (Z→A). Todas las interacciones son completamente dinámicas, sin ningún botón de enviar, usando el evento onChange de cada control.

Componentes reutilizables (Punto 4)

La aplicación cuenta con 8 componentes reutilizables funcionales, organizados en la carpeta src/components/:

1. **Header** — Cabecera con el título de la aplicación
2. **Footer** — Pie de página
3. **Navigation** — Barra de navegación sticky con links y ThemeToggle
4. **ThemeToggle** — Botón para cambiar entre tema claro y oscuro
5. **SearchBar** — Barra de búsqueda, filtro y ordenación reutilizable
6. **StatusMessage** — Mensajes de carga, error y lista vacía
7. **CharacterCard** — Tarjeta visual de cada personaje
8. **InfoRow** — Fila de información etiqueta-valor en la vista de detalle

Las páginas se organizan en src/pages/ y las funciones de acceso a la API en src/utils/api.ts. Los tipos TypeScript se definen en src/types/index.ts.

2.2 Funcionalidades extra

Metodología Gitflow (Extra 1)

Durante todo el desarrollo se ha utilizado la metodología Gitflow. El proyecto parte de una rama main con el commit inicial de Vite. Se inicializó git flow con las ramas estándar (main, develop, feature/, release/, hotfix/). Todo el desarrollo se realizó en la rama feature/setup-estructura, con commits atómicos por cada funcionalidad completada. Al terminar, la feature se mergeó a develop mediante git flow feature finish, y finalmente se creará una release hacia main para la entrega.

Tema claro/oscuro con localStorage (Extra 2)

La aplicación implementa un sistema de tema claro/oscuro mediante el componente ThemeToggle. Al cambiar el tema, se añade o elimina la clase CSS dark del body, lo que activa un conjunto de variables CSS distintas (--bg-color, --card-bg, --text-color, etc.). La preferencia del usuario se persiste en localStorage bajo la clave theme, de forma que al recargar la página el tema se restaura automáticamente con el valor guardado.

3. Justificación de diseño — Leyes de Gestalt

El diseño de la interfaz aplica de forma consciente dos Leyes de Gestalt para mejorar la usabilidad y la experiencia del usuario.

3.1 Ley de Proximidad

La Ley de Proximidad establece que los elementos que están cerca entre sí son percibidos como un grupo relacionado. En la aplicación, esto se aplica en la barra de controles de búsqueda: el campo de texto, el selector de filtro y el selector de ordenación están agrupados horizontalmente con un gap uniforme entre ellos, separados visualmente del resto del contenido. El usuario percibe estos tres controles como una unidad funcional de búsqueda, sin necesidad de etiquetas adicionales que lo expliquen.

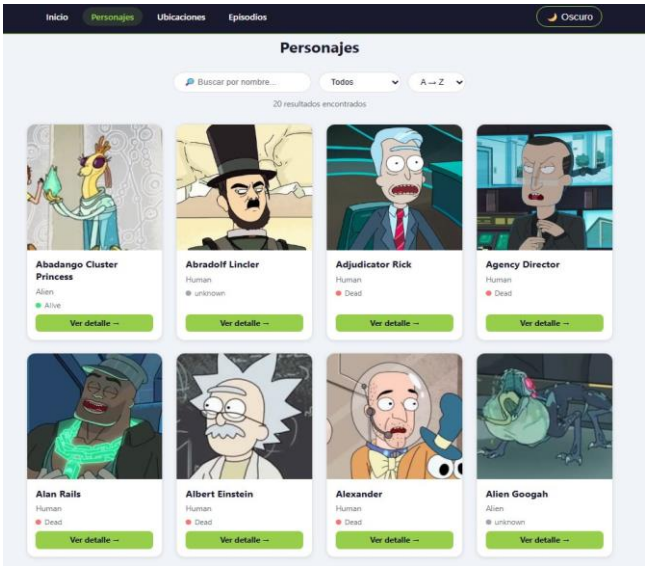
Captura — Barra de búsqueda, filtro y ordenación:

Los tres controles aparecen juntos y alineados. Su proximidad comunica al usuario que todos forman parte de la misma acción: buscar y filtrar la lista.

3.2 Ley de Similitud

La Ley de Similitud indica que los elementos con apariencia similar son percibidos como pertenecientes al mismo grupo. En la página de personajes, todas las tarjetas (CharacterCard) comparten exactamente el mismo diseño: mismas dimensiones, mismo borde, mismo border-radius, misma tipografía y misma estructura interna (imagen, nombre, especie, indicador de estado y botón). Esta repetición visual uniforme comunica al usuario que todos los elementos son del mismo tipo y tienen el mismo comportamiento al interactuar con ellos.

Captura — Grid de tarjetas de personajes:

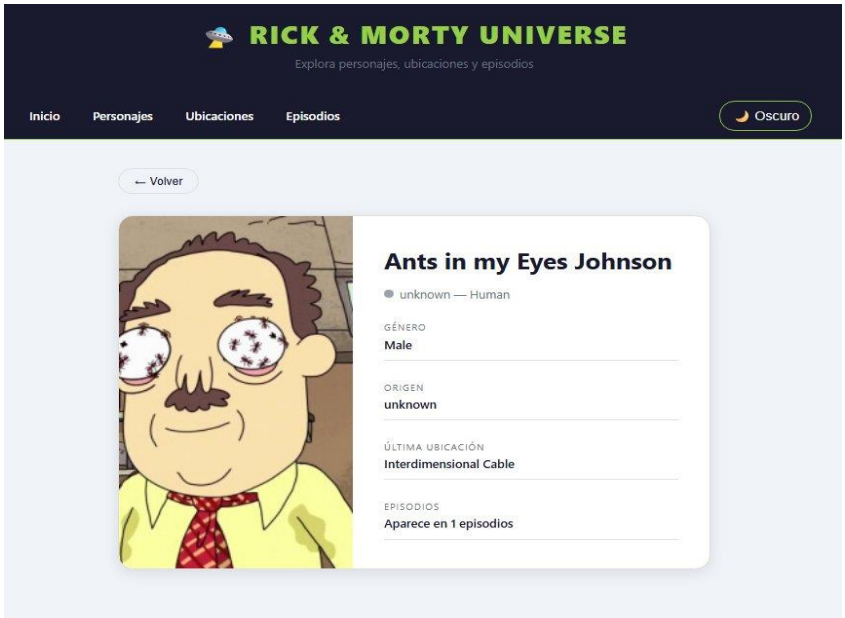


Todas las tarjetas son visualmente idénticas en estructura. El usuario identifica inmediatamente que cada tarjeta representa un personaje y sabe qué esperar al interactuar con cualquiera de ellas.

3.3 Ley de Figura-Fondo

Como ley adicional, el diseño también aplica la Ley de Figura-Fondo. En la vista de detalle de un personaje, la tarjeta de información (fondo blanco/oscuro con borde) destaca claramente sobre el fondo general de la página. La imagen del personaje actúa como figura principal y los datos textuales como información secundaria pero bien estructurada. El contraste entre el fondo de la página y el fondo de la tarjeta crea una jerarquía visual clara.

Captura — Vista de detalle de personaje:



4. Estructura del proyecto

El proyecto sigue una estructura organizada por responsabilidad:

- **src/components/** — Componentes reutilizables de UI
- **src/pages/** — Vistas completas de cada ruta
- **src/utils/** — Funciones de acceso a la API
- **src/types/** — Tipos TypeScript (Character, Location, Episode, ApiResponse)